

Weekly Report #5

Benchmarking Stream Clustering Algorithms in River

Contributor	Thinh Hai Trieu, Nguyen Trung Kien	Reporting period	29 Jun 2026 – 05 Jul 2026
Mentors	Mr. Ngo Hoang Anh	Phase	Phase 2 - HPO methodology
Repository	github.com/user/repo	Report date	September 26, 2026

1. Executive Summary

Objective - This week we scored the first HPO pipeline on five seeds, compared it with how stream-clustering papers tune their parameters, and built a revised selector from what we learned.

Status - **IN PROGRESS** On track (direction revised)

- **First direction.** For DenStream, only P2R beats the default on all three full-length datasets, by 0.008 to 0.073 ARI. On the two noisiest datasets, seed noise is as large as the gaps between methods, and CluStream uses 87% of the compute without a usable signal.
- **Literature.** Absolute parameter values do not carry over between datasets, but relative ones can. Racing spends extra seeds only on candidates still in contention, and headroom thresholds must follow the measured noise.
- **Revised direction.** We now select by headroom, race the grid with irace, and sweep λ in place of β . These changes pass a smoke test, and the full 50k run is planned for next week.

Progress tracker

Table 1: Week-4 plan versus reality, week 5.

Weekly plan item	Phase ref.	Status	Completion
N1–N2: headroom reading and selection	P2	DONE	100%
N3: fidelity check, 50k vs full length	P2	PLANNED	0%
N4: out-of-sample on 6 unseen datasets (coded)	P2	IN PROGRESS	50%
N5: DenStream screening (β settled)	P2	IN PROGRESS	40%
New: literature review; irace arms	P2	DONE	100%

Takeaway. The first pipeline is fully measured. Its weakness is the ratio of compute to signal, not a bug.

Overall: Week 5/18 - on schedule.

2. First Direction: Grid Search, One Set per Method

2.1 Pipeline

Each method ships one set per algorithm and runs it unchanged on every dataset, in these stages:

- **Grid.** Values come from the original papers, plus a relative grid with $\varepsilon = c \cdot s(D)$ and $q = \text{ratio} \cdot k$.
- **Stage A.** The whole grid runs on 50k samples of five real datasets with seed 1.
- **Arm 1.** It picks the set with the best mean ARI over the five datasets.
- **Arm 2.** It makes the same choice by leave-one-dataset-out (LODO) [5].
- **Arm 3.** It repeats Arm 2 on the relative grid.
- **P2R.** It chooses the shape by LODO and reads the granularity from a label-free curve [6].

- **Stage B.** The chosen sets run on full-length KDDCup99, Covertypes and INSECTS (seeds 2–6).

2.2 Results and cost

Table 2: DenStream, mean of seeds 2–6. Stage A scores each set on the datasets that chose it (*in-sample*) or on the one left out (*LODO*); Stage B gives full-length ARI.

Method	Stage A (50k)		Stage B ARI			Cov. samples/s
	In-sample	LODO	KDD	Cov	INS	
Default	0.045	–	0.268	0.000	0.000	72,942
Arm 1 = Arm 2	0.090	0.045	0.244	0.000	0.000	71,514
Arm 3	0.115	0.080	0.457	−0.029	0.026	23,831
P2R	0.110	0.109	0.289	0.008	0.073	292

For CluStream, no method exceeds 0.009 in Stage A or 0.038 in Stage B.

Takeaway. P2R keeps its score on unseen datasets and alone beats the default everywhere, but its gains are small and it runs 250 times longer on Covertypes. The zeros on Covertypes and INSECTS are degenerate runs: ε is too small for the data, so no cluster forms.

A five-seed round takes 1,660 runs, 6.5 hours of wall-clock time and 76.7 CPU-hours. CluStream accounts for 87% of this cost, although its best ARI on any dataset is only 0.032.

2.3 Where it falls short

P1 – Compute goes where there is no signal. CluStream’s ARI never rises above seed noise, since the spread between its configurations is only 2.5 times the noise. Within DenStream, β has no measurable effect, so a third of the shape grid was spent without return.

P2 – Gaps between methods lie inside the noise. Each set is chosen on one seed, yet one KDDCup99 configuration ranges from 0.07 to 0.41 ARI across six seeds, while methods differ by 0.05 to 0.08. The ceiling, the maximum of 126 noisy runs, is inflated by a similar amount (Table 3), and the week-4 gate values were plain round numbers.

Table 3: DenStream at 50k. Ceiling: best grid point on seed 1. Re-run: that configuration on seeds 2–6. Gain: re-run minus default. Detectable: smallest gain a five-seed test can find (Section 3.1).

Dataset	Ceiling	Re-run	Bias	sd	Gain	Detectable
KDDCup99	0.515	0.423	0.092	0.072	0.198	0.080
Shuttle	0.297	0.211	0.086	0.085	0.211	0.094
Keystroke	0.166	0.147	0.019	0.009	0.147	0.010
INSECTS	0.086	0.077	0.009	0.008	0.077	0.009
Covertypes	0.018	0.019	−0.001	0.002	0.019	0.003

Takeaway. The detectable gain varies thirtyfold between datasets, so no single ARI threshold suits all of them: 0.02 is too low for KDDCup99 and Shuttle and too high for Keystroke and INSECTS. Pooled over the four tunable datasets, headroom differences below about 0.14 are ties.

P3 – No method transfers everywhere. P2R’s label-free criterion picks the wrong level on Shuttle and loses 94% of its ceiling there, whereas Arm 3 does well on Shuttle but fails on Covertypes. Moreover, all Stage B datasets were also used for tuning, so nothing is out-of-sample yet.

3. Lessons from the Literature

Carnein et al. [1] tune each algorithm per dataset, offline, with irace on ARI. Mansalis et al. [2] vary one parameter at a time and read the plots, with one run per setting. confStream [3] tunes online with irace-inspired sampling but no racing, and IWFO [4] optimises batch centroids rather than hyperparameters. Four lessons carry over.

- L1. Relative values transfer better than absolute ones.** Carnein’s tuned DBSTREAM radius is 0.02 on Sensor but 1.95 on KDDCup99 even on standardised data, while Mansalis’s 15–20 micro-clusters per class is relative, like our $c \cdot s(D)$ and q/k . We keep the relative grid.
- L2. The metric and the ranges are design choices.** Carnein caps micro-clusters because “the ARI can be artificially improved” by adding more, and in Mansalis purity and CMM rank the algorithms in opposite orders. Our score and thresholds need an explicit basis (P2).
- L3. β is a dead parameter.** MOA and River both use it only through the product $\beta\mu$, and pinning it at 0.75 costs only 0.0004 ARI of ceiling. We therefore pin it (P1).
- L4. Racing suits offline tuning.** confStream leaves it out because it “makes it difficult to apply irace for streaming applications”. Our tuning is offline, so we race (P1, P2).

3.1 Racing and headroom

irace [7] evaluates candidates block by block and, after each block, drops those a statistical test finds worse than the best, so extra seeds go only to survivors. Following its guide [8], a block holds one seed over all tunable datasets, and we use the t-test because the default Friedman test ranks within instances and ignores any rescaling of scores [12]. The race covers exactly the grid’s candidates and minimises $-h$, where

$$h_m(d) = \frac{\text{ARI}_m(d) - \text{ARI}_{\text{default}}(d)}{\text{ARI}_{\text{best}}(d) - \text{ARI}_{\text{default}}(d)}$$

is the headroom proposed by Probst et al. [9]: 1 matches the best grid point and 0 is no better than the default. It addresses the concern of van Rijn and Hutter [10] that performance scales differ between datasets. Table 4 lists its three thresholds; the gain gate uses the detectable gain $(z_{0.95} + z_{0.80}) \text{SE} \approx 2.49 \text{SE}$, with $\text{SE} = \sqrt{(\text{sd}_{\text{best}}^2 + \text{sd}_{\text{def}}^2)/n}$.

Table 4: The three thresholds around headroom.

Threshold	Purpose	Now	Basis
Gain gate	Gain above noise	0.02	Statistical: $\text{gain}/\text{SE} \geq z_{0.95}$ on tuning seeds
Ceiling gate	Gain large enough to matter	0.05	Judgement, fixed before seeing the data [11]
Tie band	Method differs from default	1 sd	Statistical: $1.645\sqrt{2/n} \text{sd} = 1.04 \text{sd}$ at $n = 5$

Takeaway. Only the ceiling gate is a matter of judgement. Covertypes’ gain of 0.019 is statistically real ($z \approx 18$) but has no practical meaning, because class labels need not match the cluster structure [14]. The inflated ceiling in Table 3 is the selection bias described by Cawley and Talbot [13].

4. Revised Direction: Implementation and Smoke Test

We turned these lessons into four changes, three of which are implemented.

- (a) **Headroom selection** (P2). Headroom is now the default score and datasets that fail the gate no longer vote, which leaves CluStream untuned for quality. The report prints each gate beside the noise of Table 3.

- (b) **Racing** (P1, P2). Three irace arms (raw, relative and P2R) race over the same grid as the grid arms, so only the selector differs. The new code has 38 tests.
- (c) β **pinned at 0.75**, $\lambda \in \{0.25, 1, 2\}$ (P1). In a 20k pilot, $\lambda \geq 1$ gave more ARI and 2–5 times the throughput on INSECTS and Shuttle, but on a single seed.
- (d) **Noise-based gain gate and frozen thresholds** (P2, P3). This is task N2.

A smoke test on DenStream (three datasets, 2,000 samples) confirmed the mechanism: the race cut 84 candidates to 17 by the second block. On the grid, nine races needed only 574 new runs for 1,782 calls, as 68% of the results were on disk; a continuous space needed 8,078, so we race on the grid.

5. Next Week's Plan

Two decisions from the mentors are needed before the runs start.

- D1. Ceiling gate.** We chose 0.05 after seeing Coverttype. From most to least rigorous, it can be tied to batch k -means, made relative to each ceiling, or kept as a dated subjective choice.
- D2. Cost limit.** A recommended set needs a limit on how much slower than the default it may run, since P2R gains 0.008 ARI on Coverttype at 250 times the run time.

Table 5: Plan for week 6.

#	Task	Definition of done (verifiable)	Phase
N1	Stage A v3, DenStream, 50k (≈ 1.7 h)	irace-vs-grid table, new sets	P2
N2	Noise-based gain gate; ceiling gate per D1	Both recorded and dated before N3	P2
N3	Out-of-sample on 6 unseen datasets, 5 seeds	Win/tie/loss against the default	P2
N4	Fidelity check, 50k vs full length	Spearman of top-15 configurations	P2

Takeaway. N3 is the only real test of transfer. If the new sets fail to beat the default on most unseen datasets that pass the gate, we will report that ARI-based tuning does not transfer for River's DenStream on raw data.

6. References / Links

Code and notes this week. The code is in `hpo_pipeline/irace_tune.py` and `hpo_compare.py`, and the notes are in `paper-and-prompts/` (`HPO_HEADROOM_NGUONG.md`, `report.md`).

- [1] M. Carnein, D. Assenmacher, H. Trautmann. An empirical comparison of stream clustering algorithms. *CF'17*, pp. 361–366, 2017.
- [2] S. Mansalis, E. Ntoutsi, N. Pelekis, Y. Theodoridis. An evaluation of data stream clustering algorithms. *Stat. Anal. Data Min.*, 2018.
- [3] M. Carnein, H. Trautmann, A. Bifet, B. Pfahringer. confStream: automated algorithm selection and configuration of stream clustering algorithms. *LION*, 2020.
- [4] P. Thakral, Y. Kumar. An improved water flow optimizer for data clustering. *SN Comput. Sci.* 5:715, 2024.
- [5] V. Cerqueira et al. A framework for evaluating and benchmarking concept drift detection methods. *KDD*, 2026.
- [6] X. Fan, Y. Yue, P. Sarkar, Y. X. R. Wang. On hyperparameter tuning in general clustering problems. *ICML*, 2020.
- [7] M. López-Ibáñez et al. The irace package: iterated racing for automatic algorithm configuration. *Oper. Res. Perspect.* 3:43–58, 2016.
- [8] M. López-Ibáñez et al. The irace package: user guide, v4.5, §§10.6–10.7, FAQ 12.11. [CRAN](#).
- [9] P. Probst, A.-L. Boulesteix, B. Bischl. Tunability: importance of hyperparameters of machine learning algorithms. *JMLR* 20(53), 2019.
- [10] J. N. van Rijn, F. Hutter. Hyperparameter importance across datasets. *KDD*, 2018.
- [11] D. Lakens, A. M. Scheel, P. M. Isager. Equivalence testing for psychological research: a tutorial. *AMPPS* 1(2):259–269, 2018.
- [12] J. Demšar. Statistical comparisons of classifiers over multiple data sets. *JMLR* 7:1–30, 2006.
- [13] G. C. Cawley, N. L. C. Talbot. On over-fitting in model selection and subsequent selection bias in performance evaluation. *JMLR* 11:2079–2107, 2010.
- [14] I. Färber et al. On using class-labels in evaluation of clusterings. *MultiClust @ KDD*, 2010.